

**DO NOT REMOVE FROM EXAM VENUE**

## University of Plymouth

**MODULE CODE: ELEC351 Examination**

**TITLE OF PAPER:     Advanced Embedded Programming**

**TIME ALLOWED**                      Three Hours

**DATE**                                  Tuesday 17<sup>th</sup> January 2023

**TIME**                                  09.00 AM – 12.00 PM

**FACULTY**                              Science and Engineering

**SCHOOL**                               Engineering, Computing and Mathematics

**ACADEMIC YEAR**                  2022 / 2023

**STAGE**                                 Four

---

**INSTRUCTIONS TO CANDIDATES:**

Answer **ALL** questions from **Section A**. These are worth 50 marks.

Answer **TWO** questions from **Section B**. Each is worth 25 marks.

**Candidates are not permitted to look at the examination paper until instructed to do so.**

## SECTION A (Answer All Questions)

All questions are specifically related to embedded software using Mbed OS 6 and the C++ language (C++14 standard).

**Remember to give a justification for each answer you give.**

Marks for each question are indicated.

**Q1.** Consider the code below which defines a custom class `Complex`

```
#include <iostream>
using namespace std;

class Complex {
    private:
        double imag;
        double real;

    public:
        Complex(double r=0.0, double i=0.0) {
            this->imag = i;
            this->real = r;
        }
        Complex conjugate() {
            Complex y(this->real, -1.0*this->imag);
            return y;
        }
        void operator=(const Complex& u) {
            this->real = u.real;
            this->imag = u.imag;
        }
        void display() {
            cout << this->real;
            if (this->imag >= 0.0) {
                cout << " + j" << this->imag;
            } else {
                cout << " - j" << -this->imag;
            }
            cout << endl;
        }
};
```

**Listing 1 – Complex Class**

**Question 1 continued ...**

## Question 1 continued

It is proposed this class is then used as follows:

```
Complex c1(2.0, 3.0);  
Complex c2;  
c2 = c1.conjugate();  
c1.display();  
c2.display();
```

### Listing 2 – Using the Complex class

- (a) In **Listing 1**, when `c2` is declared, no parameters are provided. Does this code compile? Justify your answer. [2 marks]

- (b) Identify each occasion in **Listing 2** where the class constructor will execute. [2 marks]

- (c) The member functions in **Listing 1** refer to the keyword `this`. Using the statement `c1.display()` as an example, briefly explain the meaning of the keyword `this` and what it refers to. [3 marks]

- (d) In the absence of any compiler optimisation, how many copy operations will be performed when the following line executes?

```
c2 = c1.conjugate();
```

[2 marks]

- (e) The code below instantiates `Complex` using dynamic memory allocation.

```
Complex* c3 = new Complex(4.0, 5.0);  
c3->display();  
delete c3;  
c3 = nullptr;
```

Why do you think `c3` is set to the value `nullptr`?

[3 marks]

**Q2.** Consider the C++ template class below in **Listing 3**.

```
#include <iostream>
using namespace std;

template <class T, uint16_t N>
class Circular {
    protected:
        T buffer[N];
        uint16_t index;

    public:
        Circular() {
            index = 0;
            for (uint16_t n=0; n<N; n++) {
                buffer[n] = (T)0;
            }
        }
        virtual void push(T u) {
            buffer[index] = u;
            index++;
            if (index == N) {
                index = 0;
            }
        }
        T oldest() {
            return buffer[index];
        }
        void display() {
            for (auto n : buffer) {
                cout << n << endl;
            }
        }
};
```

### **Listing 3 – The Circular class**

This class is then used as shown in **Listing 4** below.

```
Circular<double, 3>* obj = new Circular<double, 3>();
obj->push(1.0);
obj->push(3.0);
obj->push(4.0);
obj->display();
delete obj;
```

### **Listing 4 – Using the Circular class**

**Question 2 continued ...**

## Question 2 continued

- (a) What is the meaning and purpose of the keyword `protected` in **Listing 3**?  
[2 marks]
- (b) Is the size of the array `buffer` determined at compile time or run-time? Justify your answer.  
[2 marks]
- (c) The class `Circular` is said to be a template class. Briefly explain what this means, using this code as an example.  
[2 marks]

An additional class is now added, as shown in **Listing 5**

```
template <class T, uint16_t N>
class Accumulator : public Circular<T,N> {
    private:
        T acc;
    public:
        Accumulator() {
            acc = (T)0;
        }
        virtual void push(T u) {
            acc = acc + u - this->oldest();
            Circular<T,N>::push(u);
        }
        void display() {
            cout << acc << endl;
        }
};
```

### Listing 5 – The Accumulator Class

- (d) The class `Accumulator` is said to be using class inheritance. Briefly explain what class inheritance means and how this applies to the `Accumulator` class.  
[3 marks]

Question 2 continued...

## Question 2 continued

The code below in Listing 6 makes use of the `Accumulator` class. Note that it compiles with no errors or warnings.

```
Circular<double, 3>* obj;  
obj = new Accumulator<double, 3>();  
obj->push(1.0);  
obj->push(3.0);  
obj->push(4.0);  
obj->display();  
delete obj;
```

### Listing 6 – Using the Accumulator Class

- (e) The datatype of `obj` is `Circular<double, 3>*`, but it is pointing to an object of type `Accumulator<double, 3>*`. This is perfectly valid C++. What is the name of the object orientated technique being used here?

[2 marks]

- (f) In Listing 6 there is a line that reads `obj->display()`. You will note that there is are member functions called `display()` in both the parent (`Circular`) and child (`Accumulator`) classes.

State which version is actually invoked at run-time and justify your answer.

[2 marks]

**Q3.** When working with multiple peripheral devices, techniques include rapid polling, interrupts, and multi-threaded programming.

- (a) Briefly describe rapid polling and how it can be used to manage multiple hardware devices.

[4 marks]

- (b) Explain why an interrupt-based solution might consume less power than rapid polling.

[2 marks]

**Question 3 continued...**

### Question 3 continued

- (c) Consider the case where one of the devices you need to use is the serial interface. If you wish to read a byte from a serial terminal, you can use the standard C library function `getchar()`. The documentation states that the default behaviour for this function is **blocking**.  
Briefly explain how this blocking behaviour would be a problem for rapid polling.  
[2 marks]
- (d) Briefly explain how multi-threaded programming can help overcome the problems with blocking.  
[2 marks]
- (e) One of the engineers on your project is concerned that multi-threaded programming might add too many overheads. Describe one significant overhead of using multi-threaded programming.  
[2 marks]
- (f) Briefly explain how a multi-threaded solution can help to save power.  
[3 marks]

**Q4.** Mbed OS includes Real-Time Operating System (RTOS). It has a scheduler that employs a round-robin scheduler.

- (a) Explain the purpose of a scheduler and briefly describe how it works.  
[4 marks]
- (b) If a task is said to be in the WAITING state, what is meant by this?  
[2 marks]
- (c) A scheduler is said to use a Task Control Block (TCB). Briefly explain what is meant by a TCB and the types of information stored within it.  
[4 marks]

**END OF SECTION A**

## SECTION B

Answer **TWO** questions from section B.

All questions carry the same marks.

**Q5.** Some test code is shown below in

**Listing 7.** This allows incident light on an LDR to control the brightness of an LED lamp.

The declaration for the `AutoLight` class is shown in **Listing 8**, which is a subclass of the `Analysis` class in **Listing 9**. Read each of these closely.

```
DigitalOut led1(LED1);

int main() {
    AutoLight<16> obj(AN_LDR_PIN, LED3, 250ms);
    while (true) {
        led1 = !led1;
        thread_sleep_for(500);
    }
}
```

**Listing 7.** *AN\_LDR\_PIN is the pin name for a ADC connected to a Light Dependent Resistor. LED1 and LED3 are pin names of GPIO outputs connected to LEDs.*

- (a) `AutoLight` and `Analysis` are both template classes. Describe two general benefits of using C++ templates.

[4 marks]

- (b) The object `obj` is of type `AutoLight<16>`. What does the value 16 relate to? Justify your answer

[5 marks]

- (c) The member function `readSample()` is said to be a virtual function. What is a virtual function and how does it differ from a non-virtual function?

[6 marks]

**Question 5 continued ...**



## Question 5 continued

- (d) The author of the code claims that the class `Analysis` is designed to be reusable and safe. It has a function `insertSample`, which calls two pure virtual functions `readSample` and `meanUpdate`. Explain how the pure virtual functions make this code more reusable.

[6 marks]

- (e) The `AnalogIn` class is not interrupt safe. As a result, a quality assurance engineer is concerned there may be an error in this code. Do you agree? Justify your answer.

[4 marks]

```
#include "Analysis.hpp"
#include <cmath>

template <int N>
class AutoLight : Analysis<float, N> {
    private:
        AnalogIn adc;
        PwmOut lamp;
    public:
        AutoLight(PinName adcPin, PinName ledPin, chrono::milliseconds Tsamp)
            : adc(adcPin), lamp(ledPin), Analysis<float, N>(Tsamp) {
            lamp.period_us(1000);
            lamp = 0.0;
        }
        virtual float readSample() {
            float s = adc.read();
            return s;
        }
        virtual void meanUpdate (const float newMean) {
            //Duty cycle is a function of the mean light level
            lamp=(pow(100, newMean)-1.0)/99.0;    //Set duty (0..1)
        }
};
```

**Listing 8** – The `AutoLight` class subclasses the `Analysis` class, and provides concrete implementation of the functions `readSample` and `meanUpdate`

Question 5 continued ...

## Question 5 continued

```
#include "mbed.h"
#include <chrono>

template <class T, int N>
class Analysis {
    private:
        T _buffer[N];           //Buffer of samples
        Ticker tmr;             //Ticker (multi-shot timer)
        uint16_t index;         //Array index

    void insertSample() {
        _buffer[index++] = readSample(); //Read & add sample to buffer
        if (index == N) {
            index = 0;
            double m = mean();           //Calculate the mean
            meanUpdate (m);              //Notify this has happened
        }
    }

    public:
    Analysis(chrono::milliseconds Tsamp) : index(0) {
        tmr.attach(callback(this, &Analysis::insertSample), Tsamp);
    }

    ~Analysis() {
        tmr.detach();
    }

    double mean() const { //Calculate mean
        double mean = 0.0;
        for (uint16_t n=0; n<N; n++) mean += _buffer[n];
        return mean/(double)N;
    }
    // Pure virtual functions
    virtual T readSample() = 0;
    virtual void meanUpdate (const T newMean) = 0;
};
```

### Listing 9 – The Analysis class

**Q6.** Carefully read through **Listing 10**. This uses the class `Flasher` defined in **Listing 11** and **Listing 12**.

```
int main() {
    auto eventHandler = [] (Flasher::ButtonEvent btn) {
        switch(btn) {
            case Flasher::UP:
                cout << "Up button pressed" << endl;
                break;
            case Flasher::DOWN:
                cout << "Down button pressed" << endl;
                break;
        }
    };

    Flasher pt(BTN1_PIN, BTN2_PIN, LED1, eventHandler);
    while(true) {
        sleep();
    }
}
```

**Listing 10** `main.cpp`, where `BTN1_PIN`, `BTN2_PIN` and `LED1` are pin names for GPIO pins connected to two push-switches and an LED respectively.

The class `Flasher` has been written to flash an LED and allow the flash rate to be controlled by two push switches while preventing switch bounce. The code uses both interrupts and threads, so has the Mbed OS “Real Time Operating System” (RTOS) enabled.

(a) The constructor in **Listing 11** is run when an instance of `Flasher` is created.

- (i) Does the LED flash when the code is first run (before any buttons are pressed)? Justify your answer

[2 marks]

- (ii) This code is said to be multi-threaded. In general, briefly explain what is meant by a thread and describe two benefits threads provide.

[6 marks]

- (iii) How many threads are used in this code and briefly outline the tasks they perform.

[2 marks]

**Question 6 continued ...**

## Question 6 continued

- (b) The developer of the `Flasher` class defined in defined in **Listing 11** and **Listing 12** claims it is very responsive to button presses because interrupts are used. The developer also says that problems with switch bounce have been addressed.
- (i) Is there evidence in the source code that switch bounce has been correctly addressed? Justify your answer, making reference to the source code in **Listing 11** and **Listing 12**.  
[4 marks]
- (ii) There is a property of the `Flasher` class `buttonEventQueue` of type `EventQueue`. Briefly explain the function of an event queue, how it is used in this software and why.  
[6 marks]
- (c) The developer of the `Flasher` class also says this code has no race conditions or deadlocks, and that any non-reentrant functions have been accounted for.
- (i) Which functions used in this code would you class as non-reentrant?  
[2 marks]
- (ii) Are there any race conditions in this code? Justify your answer.  
[3 marks]

Question 6 continued ...

## Question 6 continued

```
class Flasher {
public:
    typedef enum {UP, DOWN} ButtonEvent;
private:
    InterruptIn btnDown, btnUp;      //Input signals
    DigitalOut led;                  //Flashing LED
    milliseconds T = 500ms;          //Flash interval
    Mutex lockT;
    Thread flashThread, buttonThread;
    EventQueue buttonEventQueue;     //Handles button events
    function<void(ButtonEvent)> buttonEventCallback;

    void inputUP_ISR();
    void inputDOWN_ISR();
    void slowDown();
    void speedUp();
    void setButtonISR_State(ButtonEvent, bool);

public:
    Flasher(PinName upPin, PinName downPin, PinName ledPin,
            function<void(ButtonEvent)> fn={})
        : btnUp(upPin), btnDown(downPin), led(ledPin),
          flashThread(osPriorityRealtime), buttonEventCallback(fn)
    {
        // Turn on Button ISRs
        setButtonISR_State(UP, true);
        setButtonISR_State(DOWN, true);

        // Flash behaviour
        auto flash = [&]() {
            while (true) {
                led = !led;
                ThisThread::sleep_for(T);
            }
        };
        flashThread.start(flash);

        //Event queue for button events
        buttonThread.start([&]() {buttonEventQueue.dispatch_forever();});
    }

    ~Flasher() {
        setButtonISR_State(UP, false);
        setButtonISR_State(DOWN, false);
    }
};
```

## Listing 11 – Flasher.hpp

```

#include "Flasher.hpp"

void Flasher::inputUP_ISR() {
    setButtonISR_State(UP, false); //Disable ISR
    buttonEventQueue.call(callback(this, &Flasher::speedUp)); //Speed up
    buttonEventQueue.call(buttonEventCallback, UP);
    buttonEventQueue.call_in(250ms, [&]() {setButtonISR_State(UP, true);} );
}

void Flasher::inputDOWN_ISR() {
    setButtonISR_State(DOWN, false); //Slow down
    buttonEventQueue.call(callback(this, &Flasher::slowDown));
    buttonEventQueue.call(buttonEventCallback, DOWN);
    buttonEventQueue.call_in(250ms, [&]() {setButtonISR_State(DOWN, true);} );
}

void Flasher::slowDown() {
    lockT.lock();
    T += 100ms;
    lockT.unlock();
}

void Flasher::speedUp() {
    lockT.lock();
    if (T > 100ms) T -= 100ms;
    lockT.unlock();
}

void Flasher::setButtonISR_State(ButtonEvent btn, bool isEnabled = true) {
    CriticalSectionLock lock;
    switch(btn) {
        case UP:
            if (isEnabled) {
                btnUp.rise(callback(this, &Flasher::inputUP_ISR));
            } else {
                btnUp.rise(NULL);
            }
            break;
        case DOWN:
            if (isEnabled) {
                btnDown.rise(callback(this, &Flasher::inputDOWN_ISR));
            } else {
                btnDown.rise(NULL);
            }
    }
}

```

#### Listing 12 Flasher.cpp

**Q7.** A developer has written some software to control a servo (a device that rotates to a given angle). The main code is shown in

**Listing 13.** The code in **Listing 14** is a pure virtual class `ServoDevice`. As real hardware is not available, the developer has written a subclass of `ServoDevice` to mock (simulate) a real device. This is the `MockedServo` class shown in **Listing 15**. Buttons 1 and 2 turn the servo left and right. The terminal output displays status information, including the current servo position.

(a) Signal-wait is a mechanism used for thread synchronisation. Briefly explain how it is used in this code and for what purpose.

[4 marks]

(b) This code makes use of event queues.

(i) lambda functions / closures are used in combination with event queues in this code. Briefly explain what a C++ closure is and suggest any benefits they bring.

[4 marks]

(ii) The developer claims that by using event queues, mutex locks were not needed in any of the threads. Do you completely agree in this case? Give a full justification for your answer.

[8 marks]

(c) The developer also claims this code is very power efficient. Do you agree? Justify your answer.

[4 marks]

(d) The developer claims to have used a pure virtual class to make it easier to test both real and simulated hardware. Do you agree? Give a full justification for your answer.

[5 marks]

**Question 7 continued ...**

## Question 7 continued

```
void task1();
void task2();
void task3();

//Threads
Thread t1(osPriorityNormal); //Button 1
Thread t2(osPriorityNormal); //Button 2
Thread t3(osPriorityNormal); //Blue Button
osThreadId_t mainThreadID; //Main (for console output)
Thread servoEventThread(osPriorityNormal); //Servo
EventQueue servoEventQueue;

//Hardware
ServoDevice* servo;

int main() {
    mainThreadID = ThisThread::get_id();

    //Mocked hardware
    auto callBack = [] (const char* msg)->void {
        cout << msg << endl;
    };

    servo = new MockedServo(callBack);
    servoEventThread.start([&]() {servoEventQueue.dispatch_forever();});

    //Thread for switches
    t1.start(task1);
    t2.start(task2);

    while (true) {
        ThisThread::flags_wait_any(1 | 2);
        int16_t pos = servo->getServoPosition();
        cout << "Sevo Position: " << pos << endl;
    }
    delete servo;
}
```

Question 7 continued ...



## Question 7 continued

```
void task1() {
    DigitalIn sw(BTN1_PIN);
    while(true) {
        while(sw==0);
        servoEventQueue.call( [=]() {servo->rotateRight();});
        servoEventQueue.call( []() {osSignalSet(mainThreadID, 1);});
        ThisThread::sleep_for(50ms);
        while(sw==1);
        ThisThread::sleep_for(50ms);
    }
}

void task2() {
    DigitalIn sw(BTN2_PIN);
    while(true) {
        while(sw==0);
        servoEventQueue.call( [=]() {servo->rotateLeft();});
        servoEventQueue.call( []() {osSignalSet(mainThreadID, 2);});
        ThisThread::sleep_for(50ms);
        while(sw==1);
        ThisThread::sleep_for(50ms);
    }
}
```

### Listing 13 main.cpp

```
class ServoDevice {

protected:
    function<void(const char*)> ready;

public:
    ServoDevice(function<void(const char*)> f={}) : ready(f) {
    }
    virtual ~ServoDevice() {
    }
    virtual void rotateRight() = 0;
    virtual void rotateLeft() = 0;
    virtual int16_t getServoPosition() = 0;
};
```

### Listing 14 ServoDevice.hpp

Question 7 continued ...

## Question 7 continued

```
class MockedServo : public ServoDevice {
    private:
        int16_t position;

    public:
        MockedServo(function<void(const char*)> f) : ServoDevice(f) {
            position = 0;
        }
        virtual ~MockedServo() {
            position = 0;
        }

        virtual void rotateRight() override {
            ThisThread::sleep_for(500ms);    //Model delay for servo to move
            position++;
            ready("RIGHT ROTATE COMPLETE");    //Call back when done
        }
        virtual void rotateLeft() override {
            ThisThread::sleep_for(500ms);
            position--;
            ready("LEFT ROTATE COMPLETE");
        }
        virtual int16_t getServoPosition() override {
            return position;
        }
};
```

**Listing 15 MockedServo.hpp**

**END OF PAPER**